3. Add the columns or fields which you would prefer as instance variables inside the class.
4. Ensure those instance variables are private.
5. Create getter and setter methods.
6. Don't add any custom logic inside these methods.
7. Use the @PrimaryKey annotation to mark a field as the primary key
8. Use the @Ignore annotation to mark a field so that it is not considered as a column in your table.

Refer to SNIPPET 1 from HERE http://dgit.in/RealmCode

That's all you need to make a model in Realm. Each class you create which extends from RealmObject can be considered to be a separate table while visualizing the data.

Data Types

Realm supports the following field types: boolean, byte, short, ìnt, long, float, double, String, Date and byte[]. The integer types byte, short, int, and long are all mapped to the same type (long actually) within Realm. To represent relationships, Realm uses something called a RealmList which is conceptually similar to an ArrayList in Java. The RealmList can work with boxed data types such as Boolean, Byte, Short, Integer, Long, Float, Double and RealmObject itself allowing you to create a RealmList<? extends RealmObject>.

Annotations

Realm uses several annotations such as @PrimaryKey @Index @ Required @Ignore to let you define certain requirements in your data model.

@Required: Lets you mark a field so that it cannot have null values.

@Ignore: Lets you mark a field so that its value is not saved to the database. For example, you may want to use the same model class for representing JSON data with some additional fields and store only some of them in the database.

@Index: Adds a search index to the field which makes inserts slower and the data file larger but queries will be faster. It is recommended to add index when optimizing specific situations for read performance. Realm supports indexing: String, byte, short, int, long, boolean and Date fields.

@PrimaryKey: Specifying this over a field lets you mark that field to be used as a primary key by Realm. You can apply this to a field whose data type is either an integer (byte, short, int, long) or String. It is currently not possible to use a compound primary key. Primary keys cannot have a null value, and the @PrimaryKey annotation uses the @Required annotation implicitly.

Limitations

The following restrictions apply when you are creating a model class in Realm.

- Instance fields should be kept private.
- Don't write custom logic inside getter and setter methods as they won't be executed.
- Both public and private static fields are allowed along with static methods.
- You can implement interfaces without any methods in them.
- It is not possible to override methods such as toString() or equals().
- Limitations with respect to other aspects of Realm are explained in detail HERE (https://realm.io/docs/java/latest/#general)

Relationships

Let's examine our Bucket Drops app where multiple users can use the app on the same device and add drops to their bucket list. We need to track which person added the drop and use relationships for that purpose. We create a class called Person with a relationship to the Drop class which we created earlier.

Refer to SNIPPET 2 from HERE http://dgit.in/RealmCode

One Person object can be related to many Drop objects with the help of the RealmList class. Similarly, one-to-one and many-to-many relationships can be modelled easily. It can be visualized as shown below.

Person	
Email	Drops
abc@gmail.com	
xyz@gmail.com	-----

Drop			
What	Added	When	Completed
Party	...	...	...
Movie	...	...	...
Outing	...	...	...

Now that you have seen how relationships look like in Realm, lets get to the fun part where we perform some operations on this database in the next section.

Configuration: How to set it up?

CRUD: Create, Read, Update, Delete

Create

You can add your own constructors in Realm as long as you have one default constructor.

- Get an instance of the Realm class
- Begin a transaction
- Create the object
- Set the required properties whose values will be persisted
- Commit the transaction

Refer to SNIPPET 3 from HERE http://dgit.in/RealmCode

Instead of using the copyToRealm method which adds a new entry time, you can also use the copyToRealmOrUpdate method which inserts if the item does not exist and updates the item otherwise. It however requires your model to have a primary key. Another way of performing the insert would be as follows.

Refer to SNIPPET 4 from HERE http://dgit.in/RealmCode

Read

Read operations don't block each other and don't require transactions. They use a FLUENT interface with the help of which you can define conditions on which objects you would like to read.